

amf/ Code Walkthrough

Line-by-line / section-by-section explanation of every source file · 2026-07-21

How this document is organized

Every source file in `amf/` is walked through in build order: shared FSM types → uprobe target data → attach mechanics → the loader → the two BPF programs → the build system. Each code block reproduces the actual source (line numbers refer to the file as it stands today); the paragraph immediately after explains it.

One compression, stated up front so it isn't a silent shortcut: several files contain long runs of *structurally identical* blocks -- 43 near-identical `attach_target` struct literals in `amf_functions.c`, 43 near-identical 3-line uprobe handler bodies in `amf_tracer.bpf.c`, and a disabled legacy block repeating that same shape 22/23 more times. For each of those, the first instance is explained in full, then the remaining instances are presented as a compact table (which is also the more useful format for that data) rather than repeating the same explanatory paragraph dozens of times.

Not covered: `amf_tracer.skel.h` (278KB, mechanically generated by `bpftool gen skeleton` from the compiled BPF object -- not hand-authored, regenerates automatically on every build) and the compiled `.o`/binary artifacts. `amf/doc/`'s own PDFs/text files are reference material, not code.

1. sm_map.h

Shared key/value struct layout for the FSM's two BPF hash maps. Included by both the BPF side (sm_map.c) and the userspace side (amf_comm.c).

Lines 1-2 -- include guard

```
#ifndef SM_MAP_H
#define SM_MAP_H
```

Standard header guard; prevents redefinition errors if the header is transitively included more than once in the same translation unit (it is -- amf_comm.h includes it, and amf_comm.c also includes amf_comm.h).

Lines 4-23 -- doc comment

Explains the map layout (name→kind, (from,label)→to) and, critically, *why* this header is safe to use unmodified on both the BPF and userspace sides: the key/value structs are fixed-size and pointer-free (a hard requirement for any BPF map key/value type), and the __u8 etc. typedefs resolve consistently either way -- from vm_linux.h when compiled as BPF, from linux/types.h (pulled in via bpf/libbpf.h) when compiled as ordinary userspace C.

Lines 25-26 -- buffer size constants

```
#define SM_NAME_MAX 32
#define SM_LABEL_MAX 48
```

Sized with headroom over the longest actual string: the longest state name is SECURITY_POLICY_VIOLATION (26 chars, fits under 32), the longest edge label is "Nsmf Create/Update SM Context" (30 chars, fits under 48).

Lines 28-32 -- enum sm_node_kind

```
enum sm_node_kind {
    SM_KIND_NORMAL = 0, /* one of the 13 normal-path FSM states */
    SM_KIND_FAILURE = 1, /* one of the 10 failure/threat FSM states */
    SM_KIND_ENDPOINT = 2, /* external peer: UE/gNB, AMF, AUSF, UDM, PCF, SMF */
};
```

Classifies every FSM node into one of three categories. Values are pinned explicitly (=0/1/2) rather than left implicit, since this enum crosses the BPF/userspace boundary as a raw byte (struct sm_node_val.kind is __u8) -- an explicit, stable numbering avoids any risk of the two sides disagreeing on what a given byte value means.

Lines 34-49 -- the four key/value structs

```
struct sm_node_key {
    char name[SM_NAME_MAX]; /* zero-padded; must be memset before use */
};

struct sm_node_val {
    __u8 kind; /* enum sm_node_kind */
};

struct sm_edge_key {
    char from[SM_NAME_MAX]; /* zero-padded; must be memset before use */
    char label[SM_LABEL_MAX]; /* zero-padded; must be memset before use */
};

struct sm_edge_val {
    char to[SM_NAME_MAX];
};
```

sm_node_key/sm_node_val are the key/value pair for sm_nodes -- looking up a state name returns its kind. sm_edge_key is a *composite* key -- (current state, observed event label) -- and sm_edge_val's single field is the destination state; together these define sm_edges as literally a lookup table implementing the FSM's transition function. The repeated "zero-padded; must be memset before use" comments flag the single most important invariant in this whole subsystem: a BPF hash-map key is compared as raw bytes over its *entire* declared size, never as a NUL-terminated string --

if one writer zero-pads the unused tail of a 32-byte buffer and another leaves garbage there, otherwise-identical-looking keys will never match.

Lines 51-53 -- map capacity constants

```
/* amf_state_machine.py has 29 nodes / 52 edges; headroom for growth */  
#define SM_NODE_MAP_MAX_ENTRIES 64  
#define SM_EDGE_MAP_MAX_ENTRIES 128
```

Roughly 2x headroom over the actual counts (29 nodes, 52 edges), leaving room to add states/edges without touching the map definitions in sm_map.c.

Line 55 -- include guard close

```
#endif /* SM_MAP_H */
```

2. sm_map.c

Declares the two FSM hash maps and the in-kernel lookup helpers. Never compiled standalone -- `#include'd` into two separate BPF objects: `amf_tracer.bpf.c` and `amf_xdp.c`.

Lines 1-27 -- top-of-file doc comment

Establishes three facts a reader needs before anything else makes sense: (1) this file defines `sm_nodes/sm_edges`; (2) it is included into *two independently-loaded* BPF objects, not one; (3) "CO-RE" here just means "compiled against `vmlinux.h/BTF`, same toolchain as the rest of the object" -- there's no actual `BPF_CORE_READ()` field-offset relocation happening in this file, because `sm_node_key/sm_edge_key` are structs this project defines itself, not kernel structs whose layout might shift between kernel versions. It also states the map-population contract up front: a `BPF_MAP_TYPE_HASH` has no static-initializer mechanism, so both maps start empty at load and are filled at runtime by `sm_map_populate()` (`amf_comm.c`).

Lines 29-32 -- includes

```
#include "../vmlinux.h"
#include <bpf/bpf_helpers.h>

#include "sm_map.h"
```

`vmlinux.h` supplies every base type (`__u8`, `__u32`, ...) and the BPF map-type enum values from the running kernel's own BTF. `bpf_helpers.h` supplies the `SEC()` macro, `bpf_map_lookup_elem()`, and the `__uint/__type/LIBBPF_PIN_BY_NAME` map-definition helpers. `sm_map.h` brings in the four structs from Part 1.

Lines 34-50 -- sm_nodes, with the pinning comment

```
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, SM_NODE_MAP_MAX_ENTRIES);
    __type(key, struct sm_node_key);
    __type(value, struct sm_node_val);
    __uint(pinning, LIBBPF_PIN_BY_NAME);
} sm_nodes SEC(".maps");
```

The `__uint/__type` lines are libbpf's BTF-defined-map convention: this anonymous struct's only purpose is to be inspected by BTF, never actually instantiated at runtime -- `__uint(type, val)` expands to a member declared as "pointer to array of `val` ints", which encodes the constant `val` purely in the member's *type* (recoverable from BTF) without ever allocating that array. `SEC(".maps")` places the whole struct in the ELF section libbpf scans for map definitions when loading the object. The line that matters most here is the last one: `pinning = LIBBPF_PIN_BY_NAME` tells libbpf that instead of creating a private, unnamed kernel map every time this object loads, it should look for an existing pinned map at `/sys/fs/bpf/sm_nodes` and reuse it if found, or create-and-pin it there if not. Since **`amf_xdp.c` re-declares this exact same map** (via re-including this file), both objects end up sharing one underlying kernel map instead of each getting an empty private copy.

Lines 52-58 -- sm_edges

```
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, SM_EDGE_MAP_MAX_ENTRIES);
    __type(key, struct sm_edge_key);
    __type(value, struct sm_edge_val);
    __uint(pinning, LIBBPF_PIN_BY_NAME);
} sm_edges SEC(".maps");
```

Identical shape and rationale to `sm_nodes`, pinned at `/sys/fs/bpf/sm_edges`.

Lines 60-71 -- lookup-helper doc comment

Documents a subtle but load-bearing assumption: every caller of these helpers passes a *compile-time string literal* (e.g. `sm_node_lookup("REG_RECEIVED")`), not a pointer into packet or user memory. That distinction matters to the BPF

verifier: reading a literal baked into this program's own `.rodata` is an ordinary local read, never a `bpf_probe_read_*`()-worthy unsafe access. It also restates the zero-padding invariant from `sm_map.h`'s key structs, now from the writer's side: destination buffers are zeroed (`= { }`) before the bounded copy runs.

Lines 73-79 -- `sm_key_set_name`

```
static __always_inline void sm_key_set_name(char dst[SM_NAME_MAX], const char *src)
{
    int i;
    #pragma unroll
    for (i = 0; i < SM_NAME_MAX - 1 && src[i]; i++)
        dst[i] = src[i];
}
```

Copies up to `SM_NAME_MAX - 1` (31) bytes from `src` into `dst`, stopping early at a NUL byte. Because `dst` was already zero-initialized by the caller, any bytes beyond the copied length stay zero -- exactly the padding the map-key comparison needs. `#pragma unroll` forces the compiler to fully unroll this small, fixed-bound loop into a flat sequence of scalar copies; the BPF verifier reasons far more easily about a bounded, unrolled sequence than about a loop it would otherwise have to prove terminates within its instruction-count budget.

Lines 81-87 -- `sm_key_set_label`

```
static __always_inline void sm_key_set_label(char dst[SM_LABEL_MAX], const char *src)
{
    int i;
    #pragma unroll
    for (i = 0; i < SM_LABEL_MAX - 1 && src[i]; i++)
        dst[i] = src[i];
}
```

Identical logic to `sm_key_set_name`, sized for the longer (48-byte) label field. Kept as a separate function rather than parameterizing the bound, so the loop bound is a literal constant at every call site (simpler for the verifier than threading a runtime size parameter through).

Lines 89-94 -- `sm_node_lookup`

```
static __always_inline struct sm_node_val *sm_node_lookup(const char *name)
{
    struct sm_node_key key = {};
    sm_key_set_name(key.name, name);
    return bpf_map_lookup_elem(&sm_nodes, &key);
}
```

Builds a zero-initialized key on the stack, fills its one field, and looks it up. Returns the map's value pointer, or NULL if the state name isn't in `sm_nodes`.

Lines 96-101 -- `sm_transition_lookup`'s doc comment

Spells out the semantics that make this the whole subsystem's core primitive: given the state the caller *believes* an entity is in, plus the event label just observed, it returns the FSM's expected next state -- or **NULL if no such edge exists**. That NULL is deliberately framed as a first-class threat signal ("an out-of-spec transition ... before the event even reaches userspace"), not just a failed lookup to be silently ignored.

Lines 102-109 -- `sm_transition_lookup`

```
static __always_inline struct sm_edge_val *sm_transition_lookup(const char *from,
                                                                const char *label)
{
    struct sm_edge_key key = {};
    sm_key_set_name(key.from, from);
    sm_key_set_label(key.label, label);
    return bpf_map_lookup_elem(&sm_edges, &key);
}
```

Same pattern as `sm_node_lookup`, but the key has two sub-fields, each filled with the appropriately-sized helper (from `→ sm_key_set_name`, since it's a state name; label `→ sm_key_set_label`, since it's the longer label buffer). The returned `sm_edge_val*`, if non-NULL, has one field: `to`, the destination state -- what a caller should advance its own state tracking to on a match.

3. amf_functions.h

Extern declarations for the 21 FSM-node `attach_target[]` arrays that live in `amf_functions.c`, so `amf_loader.c` can reference them without seeing their definitions.

Lines 1-4 -- guard + include

```
#ifndef AMF_FUNCTIONS_H
#define AMF_FUNCTIONS_H

#include "../common.h"
```

Standard guard. Includes `../common.h` for struct `attach_target`'s definition -- every extern below is an array of that type.

Lines 6-12 -- banner comment

States the file's organizing principle: one array per FSM node, and states that only reuse an already-declared probe (16, 21) simply have no array here.

Lines 14-42 -- Part 1: 13 normal-path externs + their counts

```
extern struct attach_target reg_received_funcs[]; /* 01 */
extern struct attach_target context_lookup_funcs[]; /* 02 */
... (11 more, one per normal-path state) ...
extern struct attach_target registered_connected_funcs[]; /* 13 */

extern int reg_received_funcs_cnt;
extern int context_lookup_funcs_cnt;
... (11 more, matching names + _cnt) ...
extern int registered_connected_funcs_cnt;
```

Each extern `struct attach_target foo[];` tells the compiler "this array exists somewhere, with this element type, but I'm not defining it here" -- the actual contents (and therefore the array's real size) live in `amf_functions.c`. The unsized `[]` is legal precisely because it's a declaration, not a definition; anyone using `reg_received_funcs` (`amf_loader.c`) needs the matching `_cnt` extern to know how many elements it has, since C arrays don't carry their own length at runtime -- that's what the second block of externs is for, each one populated in `amf_functions.c` via `sizeof(arr)/sizeof(arr[0])`.

Lines 44-65 -- Part 2: 8 failure-state externs + counts, with the reuse note

```
/* 16 REPLAY_SUSPECTED and 21 PDU_SESSION_REJECTED have no array: both
 * reuse a probe already declared above (state 05 and state 11
 * respectively) -- see amf_functions.c. */

extern struct attach_target reg_rejected_funcs[]; /* 14 */
... (7 more) ...
extern struct attach_target cleanup_required_funcs[]; /* 23 */

... matching _cnt externs ...
```

Same pattern as Part 1, for the 8 failure states that add a genuinely new symbol. The comment explains why states 16 and 21 are absent from this list at all -- not an oversight, a deliberate non-duplication (see `amf_functions.c`'s matching comments at their would-be declaration sites).

Lines 67-70 -- Part 3

```
extern struct attach_target fsm_edge_funcs[];
extern int fsm_edge_funcs_cnt;
```

One more array for the Part-3 "supplementary/edge-only" hooks that don't map to a single numbered FSM state.

Lines 72-90 -- disabled legacy externs, in a plain comment

The 6 legacy array externs (`sub_funcs`, `ue_ctx_funcs`, etc.) are wrapped in an ordinary `/* ... */` comment, not `#if 0`. A header has no function bodies to preserve for later reference the way `amf_functions.c`'s disabled block does -- these are one-line forward declarations, so a plain comment is simpler and equally effective at keeping them out of compilation while

leaving them visible as documentation of what used to be hooked.

Line 92 -- guard close

```
#endif // AMF_FUNCTIONS_H
```

4. amf_functions.c

Pure data: the 21 `attach_target[]` array definitions (43 targets total) that `amf_functions.h` only declared. No functions in this file.

Lines 1-7 -- include + banner

```
#include "amf_functions.h"

/* ... banner: one attach_target[] per FSM node, func_name verified
 * against bin/amf, prog_name <=15 chars ... */
```

Includes its own header (standard practice -- lets the compiler check the definitions below match the extern declarations). The banner records the two hard constraints every entry below satisfies: `func_name` strings are verified real symbols in `bin/amf`, and `prog_name` is kept to 15 characters or fewer.

Lines 11-26 -- reg_received_funcs, explained in full (representative of all 21 arrays)

```
/* 01 REG_RECEIVED */
struct attach_target reg_received_funcs[] = {
    {
        .prog_name = "amf_ngap_ue_msg",
        .func_name = "github.com/free5gc/amf/internal/ngap.handleInitialUEMessageMain",
        .retprobe = false,
        .link = NULL,
    },
    {
        .prog_name = "amf_handle_nas",
        .func_name = "github.com/free5gc/amf/internal/nas.HandleNAS",
        .retprobe = false,
        .link = NULL,
    },
};
int reg_received_funcs_cnt = sizeof(reg_received_funcs) / sizeof(reg_received_funcs[0]);
```

A designated-initializer array literal, one `struct attach_target` per entry. Field by field: `.prog_name` is the name of the BPF program in `amf_tracer.bpf.c` that should be attached at this symbol (must match a `SEC("uprobe/...")` name exactly, or `bpf_object__find_program_by_name()` in `amf_comm.c` fails at runtime). `.func_name` is the exact Go symbol string to resolve inside the AMF binary (verified against `bin/amf` via `nm/objdump` -- see `doc/verified_attach_points.txt`). `.retprobe` selects `uprobe` (function entry, false) vs. `uretprobe` (function return, true) -- both entries here are entry probes. `.link = NULL` is a placeholder the loader fills in at attach time (`attach_programs()` in `amf_comm.c` writes the resulting `bpf_link*` back into this same field), and `detach_programs()` reads it back to know what to tear down. The trailing `int ..._cnt = sizeof(arr)/sizeof(arr[0])` line is the standard C idiom for "number of elements in this array" -- computed once at compile time, exposed so callers never need to hardcode a count that could drift from the array's actual contents.

Lines 28-389 -- the remaining 19 arrays (37 more targets), same shape

Every other array (states 02-15, 17-20, 22-23, and the Part-3 `fsm_edge_funcs`) follows the exact same 4-field struct literal + trailing `_cnt`-via-`sizeof` pattern shown above, entry by entry:

#	State	Array	prog_name	func_name (elided prefix)	ret
02	CONTEXT_LOOKUP	context_lookup_funcs[]	amf_ctx_lookup	internal/gmm.contextTransferFromOldAmf	UP
			amf_ctx_xfer	internal/sbi/consumer.(*namfService).UEContextTransferRequest	URET
03	IDENTITY_PENDING	identity_pending_funcs[]	amf_id_req	internal/gmm/message.SendIdentityRequest	UP
			amf_id_resp	internal/gmm.HandleIdentityResponse	UP

#	State	Array	prog_name	func_name (elided prefix)	ret
04	AUTH_VECTOR_PENDING	auth_vector_pending_funcs[]	amf_auth_proc	internal/gmm.AuthenticationProcedure	UP
			amf_ausf_auth	internal/sbi/consumer.(*nausfService).SendUEAuthenticationAuthenticateRequest	URET
05	NAS_AUTHENTICATING	nas_authenticating_funcs[]	amf_auth_req	internal/gmm/message.SendAuthenticationRequest	UP
			amf_auth_resp	internal/gmm.HandleAuthenticationResponse	UP
			amf_auth_fail	internal/gmm.HandleAuthenticationFailure	UP
06	NAS_SECURITY_PENDING	nas_security_pending_funcs[]	amf_sec_mode	internal/gmm.SecurityMode	UP
			amf_sec_cmd	internal/gmm/message.SendSecurityModeCommand	UP
			amf_sec_cmplt	internal/gmm.HandleSecurityModeComplete	UP
			amf_sec_reject	internal/gmm.HandleSecurityModeReject	UP
07	UDM_REGISTERING	udm_registering_funcs[]	amf_udm_comm	internal/gmm.communicateWithUDM	UP
			amf_udm_uecm	internal/sbi/consumer.(*nudmService).UeCmRegistration	URET
08	SUBSCRIPTION_LOADING	subscription_loading_funcs[]	amf_sdm_amdata	internal/sbi/consumer.(*nudmService).SDMGetAmData	URET
			amf_sdm_smfselect	internal/sbi/consumer.(*nudmService).SDMGetSmfSelectData	UP
			amf_sdm_uectx	internal/sbi/consumer.(*nudmService).SDMGetUeContextInSmfData	UP
			amf_sdm_sub	internal/sbi/consumer.(*nudmService).SDMSubscribe	UP
09	POLICY_ASSOCIATING	policy_associating_funcs[]	amf_pcf_policy	internal/sbi/consumer.(*npcfService).AMPolicyControlCreate	URET
10	UE_CONTEXT_READY	ue_context_ready_funcs[]	amf_reg_accept	internal/gmm/message.SendRegistrationAccept	UP
11	SM_CONTEXT_PENDING	sm_context_pending_funcs[]	amf_pdu_create	internal/gmm.CreatePDUSession	UP
			amf_smf_ctx_req	internal/sbi/consumer.(*nsmfService).SendCreateSmContextRequest	URET
12	INITIAL_CONTEXT_SETUP	initial_context_setup_funcs[]	amf_ics_req	internal/ngap/message.SendInitialContextSetupRequest	UP
			amf_ics_resp	internal/ngap.handleInitialContextSetupResponseMain	UP
13	REGISTERED_CONNECTED	registered_connected_funcs[]	amf_registere	internal/gmm.Registered	UP
			amf_reg_cmplt	internal/gmm.HandleRegistrationComplete	UP
14	REG_REJECTED	reg_rejected_funcs[]	amf_reg_reject	internal/gmm/message.SendRegistrationReject	UP
15	AUTH_FAILED	auth_failed_funcs[]	amf_auth_fsm	internal/gmm.Authentication	UP
17	SECURITY_FAILED	security_failed_funcs[]	amf_nas_decode	internal/nas/nas_security.Decode	UP
18	SECURITY_POLICY_VIOLATION	security_policy_violation_funcs[]	amf_sel_sec_alg	internal/context.(*AmfUe).SelectSecurityAlg	UP
19	NF_TIMEOUT	nf_timeout_funcs[]	amf_nssf_sel	internal/sbi/consumer.(*nssfService).NSSelectionGetForRegistration	URET

#	State	Array	prog_name	func_name (elided prefix)	ret
20	SLICE_REJECTED	slice_rejected_funcs[]	amf_nssai_req	internal/gmm.handleRequestedNssai	UP
			amf_5gsm_msg	internal/gmm.transport5GSMMMessage	UP
22	CONTEXT_SETUP_FAILED	context_setup_failed_funcs[]	amf_ics_fail	internal/ngap.handleInitialContextSetupFailureMain	UP
23	CLEANUP_REQUIRED	cleanup_required_funcs[]	amf_remove_ue	internal/gmm/common.RemoveAmfUe	UP
			amf_ue_remove	internal/context.(*AmfUe).Remove	UP
P3	Supplementary / edge-only	fsm_edge_funcs[]	amf_sec_ctx_ok	internal/context.(*AmfUe).SecurityContextIsValid	UP
			amf_ics_disp	internal/ngap.handlerInitialContextSetupResponse	UP
			amf_nas_disp	internal/nas.Dispatch	UP
			amf_fsm_event	util/fsm.(*FSM).SendEvent [github.com/free5gc/, not .../amf/]	UP

Lines 330-359 -- 21 SM_CONTEXT_PENDING/PDU_SESSION_REJECTED comment

```
/* 21 PDU_SESSION_REJECTED -- no array: reject branch of CreatePDUSession,
 * same symbol as state 11's amf_pdu_create (sm_context_pending_funcs). */
```

A bare comment where an array would otherwise go, at exactly the point in file order where state 21 would sit -- documents the reuse in place rather than leaving a silent gap a reader might mistake for an oversight.

Lines 392-569 -- disabled legacy block

```
/* ... NOT part of the 23-state threat-aware FSM ... */
#if 0

/* ----- Subscription API ----- */
struct attach_target sub_funcs[] = {
    {
        .prog_name = "amf_stat_ch_sub",
        .func_name =
"github.com/free5gc/amf/internal/sbi/processor.(*Processor).AMFStatusChangeSubscribeProcedure",
        .retprobe = false,
        .link = NULL,
    },
    ... 2 more entries ...
};
int sub_funcs_cnt = sizeof(sub_funcs) / sizeof(sub_funcs[0]);

... 5 more arrays (ue_ctx_funcs, callback_funcs, evnt_funcs, n1n2msg_funcs,
other_funcs), same shape, 22 entries total ...

#endif /* AMF API attach_target arrays not in FSM */
```

Unlike the header's plain-comment treatment of the same legacy data, here it's wrapped in a real preprocessor conditional -- `#if 0 ... #endif` -- because these ARE full function-body-shaped definitions (struct literals with real field values) that the author wanted preserved verbatim and byte-for-byte compilable-again-if-uncommented, not just documented in prose. `#if 0` is the standard C idiom for "disable this code but keep it syntax-checked-adjacent and easy to re-enable", distinct from deleting it outright. These 6 arrays (22 targets) hook `internal/sbi/processor.(*Processor).* SBI-service` callbacks -- a real, working probe set from an earlier iteration of this project, superseded once the FSM-node hooks above were verified against `bin/amf`, but kept here for reference (see `doc/missing_attach_points.txt` for the full gap analysis that led to this replacement).

5. amf_comm.h

Prototypes and shared types for the attach/detach mechanics + FSM-map loader implemented in amf_comm.c.

Lines 1-7 -- guard + includes

```
#ifndef AMF_COMM_H
#define AMF_COMM_H

#include <bpf/libbpf.h>
#include "../common.h"
#include "amf_tracer.skel.h"
#include "sm_map.h"
```

bpf/libbpf.h supplies struct bpf_link, LIBBPF_OPTS, and the bpf_program__attach_* declarations used in amf_comm.c. ../common.h supplies struct attach_target. amf_tracer.skel.h supplies struct amf_tracer_bpf (the generated skeleton type every function here takes a pointer to). sm_map.h is included here -- rather than separately by amf_comm.c -- purely for convenience: anything that includes amf_comm.h (i.e. amf_comm.c and amf_loader.c) transitively gets the FSM struct types too, without a second include line.

Lines 9-14 -- sm_map_populate prototype

```
/* Populate the sm_nodes / sm_edges BPF hash maps (see sm_map.c) with the
 * full 29-node / 52-edge threat-aware AMF FSM, mirrored from
 * amf/amf_state_machine.py. Call once, right after
 * amf_tracer_bpf__open_and_load(). Returns 0 on success, <0 on the first
 * bpf_map__update_elem() failure. */
int sm_map_populate(struct amf_tracer_bpf *skel);
```

Declares the function whose implementation is in amf_comm.c, with its calling contract spelled out in the comment: call exactly once, right after skeleton load, and treat any negative return as a fatal setup error.

Lines 16-22 -- attach_programs / detach_programs prototypes

```
int attach_programs(struct amf_tracer_bpf *skel,
                   const char *bin_path,
                   pid_t pid,
                   struct attach_target *targets,
                   int cnt);

void detach_programs(struct attach_target *targets, int cnt);
```

The generic uprobe attach/detach pair -- generic in the sense that any of amf_functions.c's 21 arrays can be passed as targets/cnt; the function doesn't know or care which FSM state an array corresponds to.

Lines 25-33 -- struct tp_target

```
/* One entry per syscall tracepoint to attach */
struct tp_target {
    const char *category; /* e.g. "syscalls" */
    const char *name; /* e.g. "sys_enter_connect" */
    const char *prog_name; /* BPF program name in skeleton */
    struct bpf_link *link; /* filled by attach_tracepoints */
};
```

The tracepoint analogue of struct attach_target (which lives in ../common.h since it's shared across every NF tracer in the wider repo). This one lives here instead, since tracepoint attachment is specific to this comm layer: a tracepoint identity is a (category, name) pair (e.g. syscalls/sys_enter_connect), not a resolvable userspace symbol the way a uprobe target is.

Lines 35-39 -- attach_tracepoints / detach_tracepoints prototypes

```
int attach_tracepoints(struct amf_tracer_bpf *skel,
                      struct tp_target *targets,
                      int cnt);
```

```
void detach_tracepoints(struct tp_target *targets, int cnt);
```

Mirrors the uprobe pair's shape, one level down (tp_target instead of attach_target).

Line 41 -- guard close

```
#endif
```

6. amf_comm.c

Implements everything amf_comm.h declares: the FSM-map population routine and the generic attach/detach mechanics for both uprobes and tracepoints.

Lines 1-7 -- include + section comment

```
#include <stdio.h>

#include "amf_comm.h"

/* -- sm_map population (userspace side of sm_map.c's BPF hash maps) --
 * Node/edge data transcribed 1:1 from amf/amf_state_machine.py; keep the
 * two in sync by inspection. */
```

stdio.h for snprintf/fprintf/printf, used throughout this file. The comment flags a maintenance invariant rather than an enforced one: sm_node_data[]/sm_edge_data[] below must be kept manually in sync with amf_state_machine.py -- there's no automated check, "by inspection" is the honest description.

Line 9 -- struct sm_node_def

```
struct sm_node_def { const char *name; enum sm_node_kind kind; };
```

A local (userspace-only) pair type, distinct from sm_map.h's sm_node_key/sm_node_val -- this one holds a plain C string pointer, used only to build the literal table below before it gets copied into the fixed-size map-key format.

Lines 11-43 -- sm_node_data[]

```
static const struct sm_node_def sm_node_data[] = {
    { "REG_RECEIVED", SM_KIND_NORMAL },
    ... 12 more normal states ...
    { "REGISTERED_CONNECTED", SM_KIND_NORMAL },

    { "REG_REJECTED", SM_KIND_FAILURE },
    ... 8 more failure states ...
    { "CLEANUP_REQUIRED", SM_KIND_FAILURE },

    { "UE/gNB", SM_KIND_ENDPOINT },
    { "AMF", SM_KIND_ENDPOINT },
    { "AUSF", SM_KIND_ENDPOINT },
    { "UDM", SM_KIND_ENDPOINT },
    { "PCF", SM_KIND_ENDPOINT },
    { "SMF", SM_KIND_ENDPOINT },
};
```

All 29 FSM nodes as (name, kind) pairs, grouped visually (blank lines only, no semantic meaning) into the 13 normal states, 10 failure states, and 6 endpoints. This is the literal source-of-truth table sm_map_populate() below iterates over.

Lines 45-100 -- struct sm_edge_def + sm_edge_data[]

```
struct sm_edge_def { const char *from, *to, *label; };

static const struct sm_edge_def sm_edge_data[] = {
    { "UE/gNB", "REG_RECEIVED", "InitialUEMessage" },
    { "REG_RECEIVED", "CONTEXT_LOOKUP", "valid NAS" },
    ... 48 more edges, each a (from, to, label) triple ...
    { "CONTEXT_SETUP_FAILED", "CLEANUP_REQUIRED", "release resources" },
};
```

All 52 FSM edges, each a literal (source state, destination state, trigger label) triple -- this is the complete transition table, transcribed line-for-line from amf_state_machine.py's edge list E. Every row here becomes exactly one entry in the sm_edges BPF map once sm_map_populate() runs; sm_transition_lookup() (sm_map.c) is a lookup against precisely this table.

Lines 102-118 -- sm_map_populate: the node loop

```

int sm_map_populate(struct amf_tracer_bpf *skel)
{
    for (size_t i = 0; i < sizeof(sm_node_data) / sizeof(sm_node_data[0]); i++) {
        struct sm_node_key key = {};
        struct sm_node_val val = {};
        snprintf(key.name, sizeof(key.name), "%s", sm_node_data[i].name);
        val.kind = (__u8)sm_node_data[i].kind;

        int err = bpf_map__update_elem(skel->maps.sm_nodes,
                                       &key, sizeof(key), &val, sizeof(val),
                                       BPF_ANY);

        if (err) {
            fprintf(stderr, "sm_map_populate: sm_nodes[%s]: %d\n",
                    sm_node_data[i].name, err);
            return err;
        }
    }
}

```

Iterates `sm_node_data[]` (the loop bound is the same `sizeof`-division idiom used for every `_cnt` in this codebase). Per entry: a zero-initialized key/val pair (= `{}` -- the same zero-padding invariant `sm_map.h` documents, now on the writer's side), `snprintf` copies the name into the fixed-size key buffer (`snprintf` always NUL-terminates within the given size, and the surrounding zero-init already covers the rest), the enum is narrowed to `__u8` to match `sm_node_val.kind`'s declared width, and `bpf_map__update_elem()` -- libbpf's userspace map-write call -- writes the pair into the actual kernel map via `skel->maps.sm_nodes`. `BPF_ANY` means "create if absent, overwrite if present" (as opposed to `BPF_NOEXIST/BPF_EXIST`, which would fail depending on prior state) -- appropriate here since this function is meant to be safely re-run. Any single failure aborts the whole function immediately (no partial-failure recovery), after logging which node and error code.

Lines 120-136 -- sm_map_populate: the edge loop

```

for (size_t i = 0; i < sizeof(sm_edge_data) / sizeof(sm_edge_data[0]); i++) {
    const struct sm_edge_def *ed = &sm_edge_data[i];
    struct sm_edge_key key = {};
    struct sm_edge_val val = {};
    snprintf(key.from, sizeof(key.from), "%s", ed->from);
    snprintf(key.label, sizeof(key.label), "%s", ed->label);
    snprintf(val.to, sizeof(val.to), "%s", ed->to);

    int err = bpf_map__update_elem(skel->maps.sm_edges,
                                   &key, sizeof(key), &val, sizeof(val),
                                   BPF_ANY);

    if (err) {
        fprintf(stderr, "sm_map_populate: sm_edges[%s -> %s (%s)]: %d\n",
                ed->from, ed->to, ed->label, err);
        return err;
    }
}

```

Same structure as the node loop, but three `snprintf` calls (the composite key's two sub-fields plus the value's one field) instead of one. The error message names all three parts of the failing edge, since a mismatch could be in any of them.

Lines 138-142 -- success summary + return

```

printf(" sm_map: %zu nodes, %zu edges loaded\n",
       sizeof(sm_node_data) / sizeof(sm_node_data[0]),
       sizeof(sm_edge_data) / sizeof(sm_edge_data[0]));
return 0;
}

```

Reached only if every update above succeeded. Prints a one-line confirmation with the counts (computed the same `sizeof` way, not hardcoded), returns 0 for success.

Lines 144-168 -- attach_programs

```

int attach_programs(struct amf_tracer_bpf *skel, const char *bin_path, pid_t pid, struct attach_target
*targets, int cnt)

```

```

{
    for (int i = 0; i < cnt; i++) {

        // Initialize Option macro
        LIBBPF_OPTS(bpf_uprobe_opts, opts,
            .func_name = targets[i].func_name,
            .retprobe = targets[i].retprobe);

        // Search for program in the skeleton
        struct bpf_program *prog = bpf_object__find_program_by_name(skel->obj, targets[i].prog_name);
        if (!prog) {
            fprintf(stderr, "failed to find program: %s\n", targets[i].prog_name);
            return -1;
        }

        // Attach program to userspace function
        targets[i].link = bpf_program__attach_uprobe_opts(prog, pid, bin_path, 0, &opts);
        if (!targets[i].link) {
            fprintf(stderr, "failed to attach to function: %s\n", targets[i].func_name);
            return -1;
        }
    }
    return 0;
}

```

For each of the `cnt` targets: `LIBBPF_OPTS` is a libbpf convenience macro that zero-initializes a versioned options struct (`bpf_uprobe_opts`) and sets the two named fields -- it exists because libbpf's opts structs carry a `sz` field for ABI evolution, and this macro fills that in correctly so callers never have to think about struct-size versioning by hand. `bpf_object__find_program_by_name()` looks up the compiled BPF program (from the already-loaded skeleton's `obj`) whose name matches `targets[i].prog_name` -- this is where a typo in `prog_name` (or a missing `SEC("uprobe/...")` in `amf_tracer.bpf.c`) would surface as a runtime error, not a compile error. `bpf_program__attach_uprobe_opts()` does the actual attach: resolve `func_name` to an address inside `bin_path`'s symbol table, and install the uprobe/uretprobe (per `opts.retprobe`) at that address for `pid`. The offset argument (`0`) means "resolve purely by symbol name, no additional byte offset into the function". The resulting `bpf_link*` is written back into `targets[i].link` -- the same struct `amf_functions.c` initialized with `.link = NULL`. Either failure path returns `-1` immediately, leaving any already-attached entries in this batch attached (cleanup is the caller's responsibility, via `detach_programs`).

Lines 171-179 -- detach_programs

```

void detach_programs(struct attach_target *targets, int cnt)
{
    for (int i = 0; i < cnt; i++) {
        if (targets[i].link) {
            bpf_link__destroy(targets[i].link);
            targets[i].link = NULL;
        }
    }
}

```

Loops over every target, destroying and clearing any non-NULL link. The NULL check makes this safe to call on an array that's only partially attached (or not attached at all) -- it simply skips entries with nothing to tear down, and setting link back to NULL after destroying it makes the function idempotent (calling it twice is harmless).

Lines 184-207 -- attach_tracepoints

```

int attach_tracepoints(struct amf_tracer_bpf *skel,
    struct tp_target *targets,
    int cnt)
{
    for (int i = 0; i < cnt; i++) {
        struct bpf_program *prog =
            bpf_object__find_program_by_name(skel->obj, targets[i].prog_name);
        if (!prog) {
            fprintf(stderr, "attach_tracepoints: program not found: %s\n",

```

```

        targets[i].prog_name);
    return -1;
}

targets[i].link = bpf_program__attach_tracepoint(
    prog, targets[i].category, targets[i].name);
if (!targets[i].link) {
    fprintf(stderr, "attach_tracepoints: failed %s/%s\n",
        targets[i].category, targets[i].name);
    return -1;
}
printf(" [tp] %s/%s\n", targets[i].category, targets[i].name);
}
return 0;
}

```

Structurally identical to `attach_programs`, substituting `bpf_program__attach_tracepoint(prog, category, name)` for the uprobe attach call -- tracepoints are identified by a (category, name) pair rather than a resolvable symbol, so there's no equivalent of `bpf_uprobe_opts/retprobe` needed. One addition over the uprobe version: a `printf` confirmation line per successful attach, useful since there are only 6 of these (versus 43 uprobes, where per-entry console spam would be noise).

Lines 209-217 -- detach_tracepoints

```

void detach_tracepoints(struct tp_target *targets, int cnt)
{
    for (int i = 0; i < cnt; i++) {
        if (targets[i].link) {
            bpf_link__destroy(targets[i].link);
            targets[i].link = NULL;
        }
    }
}

```

Byte-for-byte the same logic as `detach_programs`, over `struct tp_target` instead of `struct attach_target`.

7. amf_loader.c

The tracer's process entry point. Discovers the AMF process, loads the BPF skeleton, attaches every probe, and drains the ring buffer until interrupted.

Lines 1-14 -- includes

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <bpf/libbpf.h>

#include "amf_tracer.skel.h"
#include "../events.h"
#include "../common.h"
#include "amf_functions.h"
#include "amf_comm.h"
```

Standard C + libbpf headers, plus the generated skeleton (struct `amf_tracer_bpf` and its open/load/attach/destroy functions), the shared event schema, `common.h` (attach_target, find_nf, get_nf_cgrouop_id), `amf_functions.h` (the 21 array externs), and `amf_comm.h` (attach/detach + sm_map_populate, plus tp_target).

Lines 16-17 -- stop flag + signal handler

```
static volatile sig_atomic_t stop = 0;
static void handle_signal(int sig) { (void)sig; stop = 1; }
```

`sig_atomic_t` is the C-standard type guaranteed safe to read/write from a signal handler without a data race; `volatile` stops the compiler from caching its value across the main loop's iterations (without it, an optimizing compiler could hoist the read of `stop` out of the loop entirely, since nothing in the loop body it can see modifies it). `(void)sig;` silences an unused-parameter warning -- the handler genuinely doesn't need to distinguish SIGINT from SIGTERM, both just mean "stop".

Lines 21-29 -- syscall_tps[] and its count

```
static struct tp_target syscall_tps[] = {
    { "syscalls", "sys_enter_connect", "trace_connect_enter", NULL },
    { "syscalls", "sys_exit_connect", "trace_connect_exit", NULL },
    { "syscalls", "sys_enter_accept4", "trace_accept_enter", NULL },
    { "syscalls", "sys_exit_accept4", "trace_accept_exit", NULL },
    { "syscalls", "sys_enter_write", "trace_write_enter", NULL },
    { "syscalls", "sys_exit_read", "trace_read_exit", NULL },
};
static int syscall_tps_cnt = sizeof(syscall_tps) / sizeof(syscall_tps[0]);
```

6 tracepoints, each pairing a (category, name) with the BPF handler that should run when it fires -- the connect/accept4/write/read enter-or-exit pairs that `amf_tracer.bpf.c`'s Section 2 implements. Note only *enter* is hooked for write and only *exit* for read (asymmetric on purpose -- see that file's handlers for why: the byte count is known at a different point for each syscall).

Lines 33-42 -- fmt_ts

```
static void fmt_ts(uint64_t ns, char *buf, size_t len)
{
    uint64_t s = ns / 1000000000ULL;
    uint64_t us = (ns % 1000000000ULL) / 1000;
    snprintf(buf, len, "%02u:%02u:%02u.%06llu",
        (unsigned)(s / 3600) % 24,
        (unsigned)(s / 60) % 60,
        (unsigned) s % 60,
        (unsigned long long)us);
}
```

Converts a nanosecond count into whole seconds (s) and a microsecond remainder (us), then formats HH:MM:SS. uuuuuu by taking s modulo 3600/60/60 for hours/minutes/seconds. The input, e->ts, ultimately comes from bpf_ktime_get_ns() in the BPF program -- boot-relative monotonic time, not wall-clock -- so this prints elapsed time since boot (mod 24h), not the actual time of day.

Lines 44-49 -- fmt_ipv4

```
static void fmt_ipv4(uint32_t addr_net, uint16_t port, char *buf, size_t len)
{
    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &addr_net, ip, sizeof(ip));
    snprintf(buf, len, "%s:%u", ip, port);
}
```

inet_ntop converts the raw network-byte-order 32-bit address into dotted-quad text; the port (already host-byte-order by the time it reaches here -- see read_sockaddr_in in amf_tracer.bpf.c) is appended as "ip:port".

Lines 54-95 -- handle_event

```
static int handle_event(void *ctx, void *data, size_t data_sz)
{
    (void)ctx;
    const struct event *e = data;
    char ts[24], addr[48];
    fmt_ts(e->ts, ts, sizeof(ts));

    switch (e->type) {

    case EVT_API_CALL:
        printf("[%s] %-4s API %-24s pid=%-6u tid=%u\n",
            ts, e->nf, e->api, e->pid, e->tid);
        break;

    case EVT_CONNECT:
        fmt_ipv4(e->dest_ip, e->dest_port, addr, sizeof(addr));
        printf("[%s] %-4s CONNECT -> %-22s pid=%u\n",
            ts, e->nf, addr, e->pid);
        break;

    case EVT_ACCEPT:
        fmt_ipv4(e->dest_ip, e->dest_port, addr, sizeof(addr));
        printf("[%s] %-4s ACCEPT <- %-22s pid=%u fd=%lld\n",
            ts, e->nf, addr, e->pid, (long long)e->ret);
        break;

    case EVT_WRITE:
        printf("[%s] %-4s WRITE %-6llu bytes pid=%u\n",
            ts, e->nf, (unsigned long long)e->bytes, e->pid);
        break;

    case EVT_READ:
        printf("[%s] %-4s READ %-6llu bytes pid=%u\n",
            ts, e->nf, (unsigned long long)e->bytes, e->pid);
        break;

    default:
        printf("[%s] %-4s UNKNOWN type=%u\n", ts, e->nf, e->type);
        break;
    }
    return 0;
}
```

This is the callback ring_buffer__poll() invokes once per event the kernel side submits. (void)ctx; discards the unused opaque context parameter (required by libbpf's callback signature but unneeded here, since there's only one ring buffer). data is reinterpreted as const struct event * -- the shared wire format from events.h -- and the timestamp is pre-formatted once before the switch, since every case uses it. The switch (e->type) then formats each event kind


```

        fprintf(stderr, "error: amf_cgroup_map update: %d\n", err);
        goto cleanup;
    }
}

/* 4b. Populate the threat-aware FSM (sm_nodes / sm_edges hash maps) */
printf("[*] Loading state machine...\n");
if (sm_map_populate(skel) < 0) {
    err = 1;
    goto cleanup;
}

```

Step 4 writes the resolved cgroup ID into `amf_cgroup_map` at index 0 -- this is what lets `is_amf_cgroup()` (`amf_tracer.bpf.c`) filter the system-wide socket tracepoints down to just this one container. The braces around this block are a scoping choice, not a control-flow construct -- they keep the local key variable's lifetime contained. Note the first appearance of the `goto cleanup;` pattern: from here on, every failure path jumps to a single shared cleanup label rather than duplicating teardown logic at each return point. **Step 4b** calls `sm_map_populate()` (`amf_comm.c`) to load the FSM graph; any negative return is treated as fatal.

Lines 157-200 -- main(): step 5 (43 uprobes) and the disabled legacy block

```

/* 5. Attach threat-aware AMF FSM uprobes ... */
printf("[*] Attaching state-machine uprobes...\n");
if (attach_programs(skel, exe_path, pid, reg_received_funcs, reg_received_funcs_cnt) < 0 ||
    attach_programs(skel, exe_path, pid, context_lookup_funcs, context_lookup_funcs_cnt) < 0 ||
    ... 19 more attach_programs() calls, one per array ...
    attach_programs(skel, exe_path, pid, fsm_edge_funcs, fsm_edge_funcs_cnt) < 0) {
    err = 1;
    goto cleanup;
}

/* ... disabled: the old (non-FSM) attach call ... */
#ifdef 0
...
#endif

```

One `attach_programs()` call per FSM-node array (21 total), chained with `||` inside a single `if`. Because `||` short-circuits, this attaches arrays in order and stops at the first failure -- exactly like the individual function's own early-return-on-first-failure behavior, just composed across all 21 groups instead of one. Any failure sets `err = 1` and jumps to cleanup, which (as shown further down) detaches whatever subset of the 21 groups did succeed before the failing one. The disabled `#if 0` block right after it is the historical attach call for the 6 legacy (non-FSM) arrays -- inert, kept for reference, matching `amf_functions.c`'s own disabled block.

Lines 202-232 -- main(): steps 6-7

```

/* 6. Attach syscall tracepoints */
printf("[*] Attaching tracepoints...\n");
if (attach_tracepoints(skel, syscall_tps, syscall_tps_cnt) < 0) {
    err = 1;
    goto cleanup;
}

/* 7. Ring buffer + event loop */
rb = ring_buffer_new(bpf_map__fd(skel->maps.events),
                    handle_event, NULL, NULL);
if (!rb) {
    fprintf(stderr, "error: ring_buffer_new failed\n");
    err = 1;
    goto cleanup;
}

signal(SIGINT, handle_signal);
signal(SIGTERM, handle_signal);

```

```

printf("\n[*] Tracing AMF (pid=%d). Ctrl+C to stop.\n\n", pid);
printf("%-4s %s\n", "NF", "API");
printf("%-4s %s\n", "----", "-----");

while (!stop) {
    err = ring_buffer_poll(rb, 100);
    if (err < 0 && err != -EINTR) {
        fprintf(stderr, "ring_buffer_poll: %d\n", err);
        break;
    }
    err = 0;
}
}

```

Step 6 attaches all 6 syscall tracepoints in one call. **Step 7** creates a userspace ring_buffer wrapper around the kernel events map's file descriptor, registering handle_event as the per-event callback (the two trailing NULLs are the opaque context pointer passed through to the callback, and a sample-rate hint -- both unused here). Signal handlers are installed only now, right before the blocking loop begins -- installing them earlier would risk stop being set before rb even exists. The loop itself calls ring_buffer_poll(rb, 100) (100ms timeout) repeatedly until stop becomes true; a negative return is only treated as a real error if it isn't -EINTR (which is the expected, harmless result of a signal arriving mid-poll -- explicitly tolerated rather than treated as failure, since that's exactly how Ctrl-C is supposed to interrupt this call).

Lines 234-270 -- cleanup

```

cleanup:
    printf("\n[*] Detaching...\n");
    ring_buffer_free(rb);
    detach_tracepoints(syscall_tps, syscall_tps_cnt);
    detach_programs(reg_received_funcs, reg_received_funcs_cnt);
    ... 19 more detach_programs() calls, one per array, mirroring step 5 ...
    detach_programs(fsm_edge_funcs, fsm_edge_funcs_cnt);
#ifdef ...
    ... disabled legacy detach calls ...
#endif
    amf_tracer_bpf__destroy(skel);
    return err;
}

```

Every goto cleanup above lands here, and normal completion (Ctrl-C, loop exits cleanly) falls through into it too. Order: free the ring buffer wrapper, detach the 6 tracepoints, detach all 21 uprobe groups (the disabled legacy detach block sits here too, inert), then amf_tracer_bpf__destroy(skel) tears down the skeleton -- unloading every BPF program and map (except the pinned sm_nodes/sm_edges, which persist at /sys/fs/bpf across process exits by design). Every detach function tolerates being called on a never-attached or partially-attached array (NULL-checked internally), so this unconditional full teardown is safe regardless of how far setup actually got. Returns err -- 0 for a clean Ctrl-C exit, 1 for any setup failure that jumped here early.

8. amf_tracer.bpf.c

The uprobe/tracepoint BPF program itself -- everything here runs in the kernel, attached either to a specific userspace symbol (uprobes) or a kernel tracepoint.

Lines 1-19 -- top comment + includes

```
// amf_tracer.bpf.c
//
// Probe class:
//
// UPROBES - attached by the loader to specific AMF Go symbols.
// Fire only for the target PID; no extra cgroup filter needed.
// Emit EVT_API_CALL events.

#include "../vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "../events.h"

/* ... #include "sm_map.c" ... */
#include "sm_map.c"
```

The header comment states the key property of uprobes used throughout this file: because a uprobe is attached to one specific (pid, symbol) pair, every uprobe handler below fires *only* for the AMF process -- unlike the tracepoint handlers later in this file, none of the 43 uprobe handlers need to call `is_amf_cgroup()`. Includes: `vmlinux.h` (base types + kernel struct layouts, e.g. `struct sockaddr_in`), `bpf_helpers.h` (SEC, map macros, most `bpf_*` helpers), `bpf_tracing.h` (PT_REGS_* macros for reading uprobe register-based arguments -- declared but not currently used, since every handler here just records that a symbol fired), `bpf_core_read.h` (CO-RE field-relocation helpers, also unused directly in this file but pulled in for parity with other BPF sources in the project). `events.h` supplies `struct event` and the `EVT_*` constants. The `#include "sm_map.c"` is what brings `sm_nodes/sm_edges` and the two lookup helpers into this compilation unit -- textually pasted in at this point, exactly like any other `#include`.

Line 21 -- LICENSE

```
char LICENSE[] SEC("license") = "GPL";
```

A specially-named global the verifier checks before allowing this program to use any GPL-only helper (e.g. `bpf_ringbuf_reserve`, `bpf_get_current_cgroup_id`). Without this exact symbol, present and set to a GPL-compatible string, the kernel refuses to load the object at all.

Lines 26-56 -- the four maps

```
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 24); /* 16 MB */
} events SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);
} amf_cgroup_map SEC(".maps");

struct _connect_args { __u64 fd; struct sockaddr *addr; };
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 4096);
    __type(key, __u64); /* tid */
    __type(value, struct _connect_args);
} connect_scratch SEC(".maps");
```

```

struct _accept_args { __u64 fd; struct sockaddr *addr; };
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 4096);
    __type(key, __u64);
    __type(value, struct _accept_args);
} accept_scratch SEC(".maps");

```

events is a 16MB ring buffer (1 << 24 bytes) -- the single channel every event this program emits flows through to userspace. `amf_cgroup_map` is a 1-entry array holding the AMF container's cgroup ID, written once by the loader (main() step 4) and read by `is_amf_cgroup()` below. `connect_scratch/accept_scratch` are both small hash maps keyed by thread ID, existing purely to smuggle a syscall's entry-time arguments (fd, sockaddr pointer) across to its matching exit-time tracepoint -- the kernel fires enter and exit as two separate, otherwise-stateless probe invocations, so anything the exit handler needs from entry has to be stashed somewhere in between.

Lines 61-75 -- new_event

```

static __always_inline struct event *new_event(__u8 type)
{
    struct event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
    if (!e)
        return NULL;
    __builtin_memset(e, 0, sizeof(*e));
    e->type = type;
    e->ts = bpf_ktime_get_ns();
    __u64 pt = bpf_get_current_pid_tgid();
    e->pid = (__u32)(pt >> 32);
    e->tid = (__u32)pt;
    e->cid = bpf_get_current_cgroup_id();
    __builtin_memcpy(e->nf, "AMF", 4);
    return e;
}

```

The shared constructor every event-emitting handler in this file calls first. `bpf_ringbuf_reserve()` claims `sizeof(struct event)` bytes of space in the ring buffer without yet making it visible to userspace readers -- reservation and submission are separate steps precisely so a handler can fill in fields after reserving, before committing. A NULL return (buffer full) is handled by returning NULL up to the caller, which every caller checks. On success, the whole struct is zeroed first (so any field a specific event type doesn't set, e.g. address fields on an `API_CALL` event, reads back as 0/empty rather than stale ring-buffer memory from a previous use of that slot), then the fields common to every event type are filled: type (the caller's argument), ts (boot-relative nanosecond timestamp), pid/tid (the high and low 32 bits of `bpf_get_current_pid_tgid()`'s packed 64-bit return -- the kernel's own convention for returning both values from one call), cid (current cgroup ID), and nf (hardcoded to "AMF", 4 bytes including the NUL, since this file only ever traces one NF). The event is returned still unsubmitted; every caller fills in type-specific fields and then calls `bpf_ringbuf_submit()` itself.

Lines 78-85 -- is_amf_cgroup

```

static __always_inline int is_amf_cgroup(void)
{
    __u32 key = 0;
    __u64 *stored = bpf_map_lookup_elem(&amf_cgroup_map, &key);
    if (!stored || *stored == 0)
        return 0;
    return bpf_get_current_cgroup_id() == *stored;
}

```

Reads the single stored cgroup ID back out of `amf_cgroup_map[0]`. If the map entry doesn't exist yet or was never set (still 0 -- e.g. the loader hasn't reached step 4 yet, or never ran), returns false rather than risking a false match against an uninitialized value. Otherwise compares the calling process's current cgroup ID against the stored one. This function is called only by the tracepoint handlers in Section 2 -- uprobes don't need it, since they're already pid-scoped by how they're attached.

Lines 88-101 -- read sockaddr in

```
static __always_inline void read_sockaddr_in(struct event *e,
                                             struct sockaddr *sa)
{
    __u16 family = 0;
    bpf_probe_read_user(&family, sizeof(family), &sa->sa_family);
    if (family != 2 /* AF_INET */)
        return;

    e->af = 2;
    struct sockaddr_in sin = {};
    bpf_probe_read_user(&sin, sizeof(sin), sa);
    e->dest_ip = sin.sin_addr.s_addr;
    e->dest_port = __builtin_bswap16(sin.sin_port);
}
```

Unlike `new_event`'s fields (all derived from BPF helpers reading kernel-internal state), `sa` here is a pointer into the calling process's *user* memory (the `sockaddr` argument to `connect()/accept4()`), so every read goes through `bpf_probe_read_user()` rather than a plain dereference -- the verifier requires this explicit, checked-copy helper for any access to memory the kernel didn't hand the program directly. It first peeks just the `sa_family` field to confirm this is an IPv4 address (`AF_INET = 2`); anything else returns early, leaving the event's address fields at whatever `new_event`'s zeroing left them. For an IPv4 address, it re-reads the full `sockaddr_in` structure in one call, copies the raw address into `e->dest_ip` (staying in network byte order -- `fmt_ipv4` in `amf_loader.c` expects that), and byte-swaps the port into host order for `e->dest_port`.

Lines 106-135 -- one representative uprobe handler, in full (representative of all 43)

```
SEC("uprobe/amf_ngap_ue_msg")
int amf_ngap_ue_msg(struct pt_regs *ctx)
{
    struct event *e = new_event(EVT_API_CALL);
    if (!e) return 0;
    __builtin_memcpy(e->api, "REG:NGAPInitialUEMsg", 20);
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

`SEC("uprobe/amf_ngap_ue_msg")` names this program's ELF section; `amf_comm.c`'s `bpf_object__find_program_by_name()` looks it up by the matching C function name (`amf_ngap_ue_msg`) -- SEC name and function name are identical by convention throughout this file, which is what lets `amf_functions.c`'s `prog_name` strings double as both. `struct pt_regs *ctx` is the standard uprobe handler signature (gives access to the target function's register state / arguments, unused by this simple handler). Every one of the 43 active handlers has the identical 3-line body: reserve an event tagged `EVT_API_CALL`, bail out (returning 0, not an error code -- BPF programs always return an integer the attach type interprets, and 0 is the conventional "nothing to report" here) if the ring buffer was full, `memcpy` a fixed "<STATE>:<Symbol>" label into `e->api` (the exact byte count must match the literal's length -- these counts were verified programmatically when the handlers were written, since a mismatch here would silently corrupt or truncate the label), then submit. No argument or return-value inspection happens in any of them yet.

Lines 137-588 -- the remaining 42 active handlers, same shape

Grouped identically to `amf_functions.c`'s arrays (same order, same state numbering), each a byte-for-byte copy of the pattern above with only the SEC name/function name/label differing:

#	State	Function (=SEC name)	e->api label
02	CONTEXT_LOOKUP	amf_ctx_lookup	CTXLOOKUP:XferOldAmf
		amf_ctx_xfer	CTXLOOKUP:XferReq
03	IDENTITY_PENDING	amf_id_req	IDPENDING:SendReq
		amf_id_resp	IDPENDING:HandleResp
04	AUTH_VECTOR_PENDING	amf_auth_proc	AVPENDING:AuthProc

#	State	Function (=SEC name)	e->api label
		amf_ausf_auth	AVPENDING:AusfAuth
05	NAS_AUTHENTICATING	amf_auth_req	NASAUTH:SendReq
		amf_auth_resp	NASAUTH:HandleResp
		amf_auth_fail	NASAUTH:HandleFail
06	NAS_SECURITY_PENDING	amf_sec_mode	SECPENDING:Mode
		amf_sec_cmd	SECPENDING:Cmd
		amf_sec_cmplt	SECPENDING:Complete
		amf_sec_reject	SECPENDING:Reject
07	UDM_REGISTERING	amf_udm_comm	UDMREG:Communicate
		amf_udm_uecm	UDMREG:UeCmReg
08	SUBSCRIPTION_LOADING	amf_sdm_amdata	SUBLOAD:AmData
		amf_sdm_smfsele	SUBLOAD:SmfSel
		amf_sdm_uectx	SUBLOAD:UeCtx
		amf_sdm_sub	SUBLOAD:Sub
09	POLICY_ASSOCIATING	amf_pcf_policy	POLICY:AmCreate
10	UE_CONTEXT_READY	amf_reg_accept	UECTXREADY:Accept
11	SM_CONTEXT_PENDING	amf_pdu_create	SMPENDING:PduCreate
		amf_smf_ctx_req	SMPENDING:SmfCtxReq
12	INITIAL_CONTEXT_SETUP	amf_ics_req	ICS:Req
		amf_ics_resp	ICS:RespMain
13	REGISTERED_CONNECTED	amf_registered	REGCONN:Registered
		amf_reg_cmplt	REGCONN:Complete
14	REG_REJECTED	amf_reg_reject	REGREJECT:Send
15	AUTH_FAILED	amf_auth_fsm	AUTHFAILED:FsmEvt
17	SECURITY_FAILED	amf_nas_decode	SECFAILED:Decode
18	SECURITY_POLICY_VIOLATION	amf_sel_sec_alg	SECPOLICY:SelAlg
19	NF_TIMEOUT	amf_nssf_sel	NFTIMEOUT:NssfSel
20	SLICE_REJECTED	amf_nssai_req	SLICEREJ:NssaiReq
		amf_5gsm_msg	SLICEREJ:5gsmMsg
22	CONTEXT_SETUP_FAILED	amf_ics_fail	ICSFAILED:Main
23	CLEANUP_REQUIRED	amf_remove_ue	CLEANUP:RemoveUe
		amf_ue_remove	CLEANUP:UeRemove
P3	Supplementary / edge-only	amf_sec_ctx_ok	EDGE:SecCtxValid
		amf_ics_disp	EDGE:IcsDispatch
		amf_nas_disp	EDGE:NasDispatch
		amf_fsm_event	EDGE:FsmSendEvent

Lines 591-830 -- disabled legacy uprobe handlers

```

/* ... AMF API uprobes - NOT part of the 23-state threat-aware FSM ... */
#ifdef 0

SEC("uprobe/amf_stat_ch_sub")
int amf_stat_ch_sub(struct pt_regs *ctx)
{
    struct event *e = new_event(EVT_API_CALL);

```

```

    if (!e) return 0;
    __builtin_memcpy(e->api, "AMFStatusChangeSubscribe", 24);
    bpf_ringbuf_submit(e, 0);
    return 0;
}

... 22 more handlers, identical shape, matching the disabled arrays in
    amf_functions.c ...

#endif /* AMF API uprobes not in FSM */

```

Same 3-line-body pattern as the active handlers, for the 23 legacy SBI-processor probes -- inert inside `#if 0`, matching `amf_functions.c`'s disabled arrays one-for-one by `prog_name`.

Lines 833-937 -- Section 2: socket syscall tracepoints

Six handlers, genuinely distinct from each other (unlike the uprobe handlers above) -- each walked through fully:

trace_connect_enter

```

SEC("tracepoint/syscalls/sys_enter_connect")
int trace_connect_enter(struct trace_event_raw_sys_enter *ctx)
{
    if (!is_amf_cgroup()) return 0;
    __u64 tid = (__u32)bpf_get_current_pid_tgid();
    struct _connect_args args = {
        .fd = (int)ctx->args[0],
        .addr = (struct sockaddr *)ctx->args[1],
    };
    bpf_map_update_elem(&connect_scratch, &tid, &args, BPF_ANY);
    return 0;
}

```

First line in every Section-2 handler: the cgroup filter, since this tracepoint fires for every process on the system, not just the AMF. `ctx->args[]` is the raw syscall-argument array a `sys_enter_*` tracepoint provides; for `connect(2)`, `args[0]` is the file descriptor and `args[1]` is the `sockaddr` pointer. These are stashed in `connect_scratch`, keyed by the current thread ID (the low 32 bits of `pid_tgid`), so the matching exit-side handler can retrieve them.

trace_connect_exit

```

SEC("tracepoint/syscalls/sys_exit_connect")
int trace_connect_exit(struct trace_event_raw_sys_exit *ctx)
{
    if (!is_amf_cgroup()) return 0;
    __u64 tid = (__u32)bpf_get_current_pid_tgid();
    struct _connect_args *a = bpf_map_lookup_elem(&connect_scratch, &tid);
    if (!a) return 0;
    bpf_map_delete_elem(&connect_scratch, &tid);

    /* success or EINPROGRESS (-115) for non-blocking sockets */
    if (ctx->ret != 0 && ctx->ret != -115) return 0;

    struct event *e = new_event(EVT_CONNECT);
    if (!e) return 0;
    read_sockaddr_in(e, a->addr);
    e->ret = ctx->ret;
    bpf_ringbuf_submit(e, 0);
    return 0;
}

```

Looks up the scratch entry stashed at enter; if none exists (e.g. the enter probe was filtered out, or there's a race), bails out with nothing to report. The entry is deleted immediately after lookup -- scratch entries are one-shot, not meant to persist. The return-value check treats both a clean success (0) and -115 (EINPROGRESS, the expected result for a non-blocking connect that hasn't completed yet) as "this connect is worth recording"; anything else (a real connection failure) is silently dropped. On the accepted path, a new `EVT_CONNECT` event is built, its destination address resolved via `read_sockaddr_in`

(reading from the user-memory sockaddr pointer saved at enter), the syscall's return code attached, and submitted.

trace accept enter / trace accept exit

```
SEC("tracepoint/syscalls/sys_enter_accept4")
int trace_accept_enter(struct trace_event_raw_sys_enter *ctx)
{
    if (!is_amf_cgroup()) return 0;
    __u64 tid = (__u32)bpf_get_current_pid_tgid();
    struct _accept_args args = {
        .fd = (int)ctx->args[0],
        .addr = (struct sockaddr *)ctx->args[1],
    };
    bpf_map_update_elem(&accept_scratch, &tid, &args, BPF_ANY);
    return 0;
}

SEC("tracepoint/syscalls/sys_exit_accept4")
int trace_accept_exit(struct trace_event_raw_sys_exit *ctx)
{
    if (!is_amf_cgroup()) return 0;
    __u64 tid = (__u32)bpf_get_current_pid_tgid();
    struct _accept_args *a = bpf_map_lookup_elem(&accept_scratch, &tid);
    if (!a) return 0;
    bpf_map_delete_elem(&accept_scratch, &tid);

    if (ctx->ret < 0) return 0; /* failed accept */

    struct event *e = new_event(EVT_ACCEPT);
    if (!e) return 0;
    if (a->addr)
        read_sockaddr_in(e, a->addr); /* fills peer (caller) address */
    e->ret = ctx->ret; /* new socket fd */
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

The enter side is structurally identical to `trace_connect_enter`, stashing into `accept_scratch` instead. The exit side follows the same lookup-then-delete pattern, but the success test is simpler -- `accept4()` returns the new socket's file descriptor (≥ 0) on success or a negative `errno` on failure, so `ctx->ret < 0` alone identifies failure (no `EINPROGRESS`-style special case needed). The address is only resolved if `a->addr` is non-NULL (some callers pass NULL for the peer-address output parameter when they don't want it), and `e->ret` here carries the new connection's file descriptor rather than a plain status code.

trace write enter

```
SEC("tracepoint/syscalls/sys_enter_write")
int trace_write_enter(struct trace_event_raw_sys_enter *ctx)
{
    if (!is_amf_cgroup()) return 0;
    __u64 count = (size_t)ctx->args[2];
    if (count == 0) return 0;

    struct event *e = new_event(EVT_WRITE);
    if (!e) return 0;
    e->bytes = count;
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

The only Section-2 syscall hooked purely at entry, with no matching exit handler and no scratch map. `write(2)`'s third argument (`args[2]`) is the requested byte count, already fully known at entry -- unlike `connect/accept`, there's nothing meaningful to learn only at exit (a partial write's actual byte count would need the exit-side return value to be exact, but this handler accepts the entry-time requested size as a reasonable approximation and avoids the enter/exit scratch-map bookkeeping overhead the other three pairs need). Zero-length writes are skipped as uninteresting.

trace_read_exit

```
SEC("tracepoint/syscalls/sys_exit_read")
int trace_read_exit(struct trace_event_raw_sys_exit *ctx)
{
    if (!is_amf_cgroup()) return 0;
    if (ctx->ret <= 0) return 0;

    struct event *e = new_event(EVT_READ);
    if (!e) return 0;
    e->bytes = (__u64)ctx->ret;
    e->ret = ctx->ret;
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

The mirror-image choice to `trace_write_enter`: `read(2)`'s requested buffer size (known at entry) isn't the interesting number -- the *actual* bytes read is, and that's only known once the syscall returns, so this is hooked purely at exit instead. `ctx->ret <= 0` covers both error returns (negative) and EOF (exactly 0), neither of which represents real data received, so both are skipped. On a real, positive read, both `bytes` and `ret` are set to the same value here (bytes as the semantic byte count, `ret` preserving the raw syscall return for consistency with how the other handlers use that field).

9. amf_xdp.c

XDP packet enforcement at the AMF's N2 ingress (veth5538fab). Unlike the tracer, this runs before any Go code executes, sees raw packet bytes only, and can return a verdict (PASS/DROP) directly to the kernel's packet path.

Lines 1-60 -- top comment

A long header comment establishing, in order: the numbered "main procedure" this file implements (parse → categorize → look up UE state → check the FSM → update-or-flag → rate-limit gate); the current enforcement policy (only the rate limiter drops; an FSM mismatch is observability-only for now, and exactly why -- categorization is one procedure code wide, the UE key is an approximation); the protocol motivation for treating InitialUEMessage specially (unauthenticated, cheap to send, expensive for the AMF to process -- the classic N2 flood/DoS vector, and the PDF's own stated reason for reserving XDP for this class of signal); the fail-open design principle; the sm_nodes/sm_edges sharing arrangement (pinned by sm_map.c, populated by the tracer's loader, read-only from this side); and the concrete attach/detach/verify/build commands for veth5538fab, including the load-order caveat (the tracer must run first, or this object creates+pins the FSM maps empty).

Lines 62-65 -- includes

```
#include "../vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#include "sm_map.c"
```

Same base includes as the other BPF files, plus bpf_endian.h for the bpf_ntohs/bpf_ntohl/bpf_htons byte-order helpers this file needs to parse network-order packet fields. #include "sm_map.c" here is the second of the two places this happens in the whole project -- this is the line that makes amf_xdp.o share sm_nodes/sm_edges with amf_tracer.bpf.o rather than getting its own empty copies (see sm_map.c's pinning discussion in Part 2).

Lines 67-77 -- protocol constants

```
#define ETH_P_IP 0x0800
#define IPPROTO_SCTP 132
#define SCTP_PPID_NGAP 60
#define SCTP_DATA 0

/* 3GPP TS 38.413 NGAP-Constants: id-InitialUEMessage */
#define NGAP_PROC_INITIAL_UE_MESSAGE 15
```

ETH_P_IP is the standard EtherType for IPv4. IPPROTO_SCTP (132) is SCTP's IP protocol number (as distinct from TCP=6 or UDP=17). SCTP_PPID_NGAP (60) is the SCTP Payload Protocol Identifier 3GPP reserves specifically for NGAP traffic -- this is the field that lets this program tell NGAP apart from any other protocol that happens to also ride over SCTP on the same or a different association. SCTP_DATA (0) is the SCTP chunk type for a DATA chunk, as opposed to control chunks like INIT or COOKIE-ECHO. NGAP_PROC_INITIAL_UE_MESSAGE (15) is the 3GPP-standardized procedureCode value for the InitialUEMessage procedure -- the specific comment citing TS 38.413 exists because this value isn't self-evident from the wire format alone, it's a spec-defined constant.

Lines 79-84 -- rate-limit tuning constants

```
#define RATE_WINDOW_NS (1000000000ULL) /* 1 second */
#define RATE_MAX_PER_WIN 50
```

A 1-second sliding window, 50 InitialUEMessage packets per source IP per window before the rate limiter trips. The comment on the surrounding declaration (see below) explains the reasoning: a legitimate gNB serves many UEs but not at this rate from one address.

Lines 86-101 -- packet-structure structs

```
struct sctphdr_simple {
    __u16 source;
    __u16 dest;
    __u32 vtag;
```

```

    __u32 checksum;
};

struct sctp_data_chunk {
    __u8 type;
    __u8 flags;
    __u16 length;
    __u32 ts_n;
    __u16 stream_id;
    __u16 stream_seq;
    __u32 ppid;
};

```

sctphdr_simple is SCTP's common header (RFC 4960 -- source port, destination port, verification tag, checksum; 12 bytes) -- "simple" here doesn't mean incomplete, it's the entire common header, just without any of SCTP's variable-length chunk machinery generalized. sctp_data_chunk is the DATA chunk header (RFC 4960 §3.3.1) -- type, flags, length, transmission sequence number, stream identifiers, and the payload protocol identifier; 16 bytes.

Lines 103-108 -- rate_entry

```

struct rate_entry {
    __u64 window_start_ns;
    __u32 count;
};

```

The value type for amf_xdp_rate_map below: when the current window started, and how many InitialUEMessage packets have been seen from this source since.

Line 110 -- LICENSE

```

char LICENSE[] SEC("license") = "GPL";

```

Lines 112-119 -- amf_xdp_rate_map

```

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 8192);
    __type(key, __u32); /* source IPv4, network order */
    __type(value, struct rate_entry);
} amf_xdp_rate_map SEC(".maps");

```

An LRU (least-recently-used) hash map rather than a plain hash -- once 8192 distinct source IPs are being tracked simultaneously, inserting a new one evicts the least-recently-touched entry automatically, so a spread of many distinct attacker addresses ages out old entries instead of the map simply refusing new insertions once full.

Lines 121-128 -- amf_xdp_stats

```

enum { STAT_PASS = 0, STAT_DROP = 1, STAT_STATE_INVALID = 2, STAT_MAX };
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, STAT_MAX);
    __type(key, __u32);
    __type(value, __u64);
} amf_xdp_stats SEC(".maps");

```

A tiny 3-slot array acting as a set of named counters. Using an enum for the indices (STAT_MAX conveniently evaluates to the count, 3, since it's the next value after the last named one) rather than raw numbers keeps the indices self-documenting at every bump_stat(STAT_...) call site. Deliberately an ARRAY, not a HASH -- readable with a plain bpftool map dump without needing a ring-buffer consumer process running at all.

Lines 130-135 -- bump_stat

```

static __always_inline void bump_stat(__u32 idx)
{
    __u64 *v = bpf_map_lookup_elem(&amf_xdp_stats, &idx);
    if (v)

```

```

    __sync_fetch_and_add(v, 1);
}

```

Looks up the counter slot and atomically increments it in place via `__sync_fetch_and_add` -- a GCC/clang atomic builtin the BPF backend lowers to an atomic add instruction, necessary because multiple CPUs can run this XDP program concurrently on different packets and a plain `*v += 1` would race.

Lines 137-155 -- ue state map and its section comment

```

struct ue_val {
    char state[SM_NAME_MAX]; /* current FSM node name, e.g. "REG_RECEIVED" */
    __u64 last_seen_ns;
};

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 8192);
    __type(key, __u32); /* TODO: RAN-UE-NGAP-ID; source IPv4 for now */
    __type(value, struct ue_val);
} ue_state_map SEC(".maps");

```

This is the per-entity FSM state map -- the piece that lets this file check an observed event against the shared FSM graph rather than just rate-limiting blindly. `ue_val.state` reuses `SM_NAME_MAX` from `sm_map.h`, so a value read out of this map can be passed straight into `sm_transition_lookup()` as the `from` argument. The key is explicitly flagged as a placeholder: source IPv4 collides every UE behind the same gNB or NAT address into one entry; a real per-UE key (RAN-UE-NGAP-ID) needs ASN.1 PER parsing this file doesn't do yet. Unlike `sm_nodes/sm_edges`, this map has no pinning attribute -- it's private to this object, not shared with the tracer side (yet).

Lines 157-171 -- ue_state_lookup / ue_state_set

```

static __always_inline struct ue_val *ue_state_lookup(__u32 key)
{
    return bpf_map_lookup_elem(&ue_state_map, &key);
}

static __always_inline void ue_state_set(__u32 key, const char *state, __u64 now)
{
    struct ue_val val = {};
    sm_key_set_name(val.state, state); /* bounded copy, from sm_map.c */
    val.last_seen_ns = now;
    bpf_map_update_elem(&ue_state_map, &key, &val, BPF_ANY);
}

```

The lookup is a thin wrapper. The setter reuses `sm_key_set_name()` -- imported from `sm_map.c` via the `#include` -- to copy the (compile-time-literal, in every actual call site) state name into the fixed-size, zero-initialized `val.state` buffer, the same bounded-copy pattern `sm_map.c` itself uses for map keys.

Lines 173-190 -- ngap_procedure_label

```

static __always_inline const char *ngap_procedure_label(__u16 procedure_code)
{
    switch (procedure_code) {
        case NGAP_PROC_INITIAL_UE_MESSAGE:
            return "InitialUEMessage";
        default:
            return (const char *)0; /* TODO: categorize remaining procedures */
    }
}

```

The categorization step, deliberately incomplete and marked as such: only procedure code 15 maps to a label today ("InitialUEMessage", chosen to match `sm_edge_data`'s literal edge label exactly, since that's what `sm_transition_lookup()` will be matching against). Every other procedure code falls through to the default case and returns a NULL pointer -- callers must treat that as "no verdict, this packet just isn't categorized yet", explicitly not as "invalid".

Lines 192-211 -- rate_limited

```
static __always_inline int rate_limited(__u32 src_ip, __u64 now)
{
    struct rate_entry *e = bpf_map_lookup_elem(&amf_xdp_rate_map, &src_ip);
    if (!e) {
        struct rate_entry init = { .window_start_ns = now, .count = 1 };
        bpf_map_update_elem(&amf_xdp_rate_map, &src_ip, &init, BPF_ANY);
        return 0;
    }

    if (now - e->window_start_ns > RATE_WINDOW_NS) {
        e->window_start_ns = now;
        e->count = 1;
        return 0;
    }

    e->count++;
    return e->count > RATE_MAX_PER_WIN;
}
```

Classic sliding-window counter, three cases: (1) never seen this source before -- create a fresh entry with count 1, allow. (2) seen before, but the current window has expired (more than RATE_WINDOW_NS since it started) -- reset the window in place (note this mutates *e directly, the pointer returned by bpf_map_lookup_elem for a HASH/LRU_HASH map points into the map's actual backing memory, so writes through it update the map without a separate update_elem call), allow. (3) still within the current window -- increment the count and return whether it now exceeds the threshold. The return value is an int used directly as a boolean by the caller.

Lines 213-233 -- amf_xdp_enforce: header + Ethernet/IP parsing

```
SEC("xdp")
int amf_xdp_enforce(struct xdp_md *ctx)
{
    void *data = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;

    // Ethernet + IPv4 only
    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end)
        return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;

    struct iphdr *iph = (void *) (eth + 1);
    if ((void *) (iph + 1) > data_end)
        return XDP_PASS;
    if (iph->protocol != IPPROTO_SCTP)
        return XDP_PASS;
}
```

ctx->data/ctx->data_end are the raw packet's byte range as XDP hands it over, provided as plain integers that must be cast to pointers -- this indirection through long is required by the BPF verifier's packet-pointer provenance tracking. Every subsequent struct overlay follows the same pattern: cast the current position to the struct type, then immediately bounds-check (struct + 1) > data_end (i.e. "does one whole struct's worth of bytes fit before the packet ends") before touching any field -- the verifier requires this check before every packet-derived memory access, and skipping it anywhere is a verifier rejection, not a runtime bug. eth->h_proto != bpf_htons(ETH_P_IP) is a check **absent from the older xdp_ngap.c** -- this version explicitly confirms the EtherType is IPv4 before overlaying an iphdr, rather than assuming it. Non-SCTP IP traffic is passed through immediately.

Lines 235-247 -- SCTP header + DATA chunk parsing

```
// IP header length may be > 20 bytes
void *sctp_ptr = (void *) iph + (iph->ihl * 4);
struct sctphdr_simple *sctp = sctp_ptr;
if ((void *) (sctp + 1) > data_end)
    return XDP_PASS;
```

```

struct sctp_data_chunk *chunk = (void *)(sctp + 1);
if ((void *)(chunk + 1) > data_end)
    return XDP_PASS;

if (chunk->type != SCTP_DATA)
    return XDP_PASS;
if (bpf_ntohl(chunk->ppid) != SCTP_PPID_NGAP)
    return XDP_PASS;

```

`iph->ihl` is the IP header length in 32-bit words (its value can exceed 5 if IP options are present), so the SCTP header actually starts `ihl * 4` bytes after the IP header's start, not a fixed 20 -- this is why the offset is computed rather than assumed. The SCTP common header and then the DATA chunk header are overlaid and bounds-checked in the same cast-then-check pattern as before. Two filters follow: `chunk->type` must be a DATA chunk (control chunks like INIT carry no NGAP payload), and the chunk's PPID, byte-swapped from network order, must equal the NGAP-reserved value -- this is the actual protocol demultiplexing step, the point at which "is this NGAP traffic at all" gets decided.

Lines 249-266 -- procedureCode extraction + categorization

```

// NGAP PDU: first two bytes are the procedureCode
void *ngap = (void *)(chunk + 1);
if (ngap + 16 > data_end)
    return XDP_PASS;

__u8 b0 = *(__u8 *)(ngap + 0);
__u8 b1 = *(__u8 *)(ngap + 1);
__u16 procedure_code = ((__u16)b0 << 8) | b1;

// Step 2: categorize (TODO: partial coverage, see ngap_procedure_label())
const char *label = ngap_procedure_label(procedure_code);
if (!label) {
    bump_stat(STAT_PASS);
    return XDP_PASS;
}

```

The NGAP PDU begins right after the SCTP DATA chunk header. The bounds check reserves a generous 16-byte margin even though only 2 bytes are read -- matching the same margin `xdp_ngap.c` used, headroom for future parsing that goes deeper into the PDU. `procedure_code` is reconstructed as a big-endian 16-bit value from the first two raw bytes, matching where ASN.1 PER encoding places the `procedureCode` field near the front of an NGAP `InitiatingMessage/SuccessfulOutcome/UnsuccessfulOutcome`. This value is immediately handed to `ngap_procedure_label()`; an uncategorized code counts as a pass and returns immediately -- everything from here down in the function only runs for the one procedure code currently recognized.

Lines 268-289 -- the FSM check

```

__u64 now = bpf_ktime_get_ns();
__u32 src_ip = iph->saddr;

// Step 3: this source's current FSM state (default: the FSM's own
// "UE/gNB" endpoint node, for a source we've never categorized before)
struct ue_val *ue = ue_state_lookup(src_ip);
const char *from_state = ue ? ue->state : "UE/gNB";

// Step 4: is (from_state, label) a valid edge in sm_edges?
struct sm_edge_val *edge = sm_transition_lookup(from_state, label);
if (edge) {
    ue_state_set(src_ip, edge->to, now);
} else {
    // Out-of-spec transition ...
    bpf_printk("amf_xdp: FSM mismatch src=0x%x label=%s",
        bpf_ntohl(src_ip), label);
    bump_stat(STAT_STATE_INVALID);
}

```

src_ip is pulled from the IP header (still network order -- consistent with how it's used as the map key everywhere else in this file). The source's current believed FSM state is looked up; if this source has never been categorized before, it defaults to "UE/gNB", the FSM's own entry endpoint -- a deliberate choice, not an arbitrary placeholder: it's the literal state the real graph starts every UE from. sm_transition_lookup() (from sm_map.c) is then the actual check: does an edge exist from this believed state, triggered by this label? A match advances ue_state_map to the edge's destination state -- the state tracking genuinely evolves over successive packets. No match logs a bpf_printk (visible via /sys/kernel/debug/tracing/trace_pipe) and increments STAT_STATE_INVALID, but -- critically -- does not modify ue_state_map and does not return XDP_DROP here; the ground truth is left untouched rather than being corrupted by whatever the mismatched packet claimed.

Lines 291-299 -- the enforcement gate

```
// Enforcement: rate-limit InitialUEMessage regardless of FSM verdict
// above (today's only hard drop trigger -- see file header).
if (procedure_code == NGAP_PROC_INITIAL_UE_MESSAGE &&
    rate_limited(src_ip, now)) {
    bpf_printk("amf_xdp: DROP InitialUEMessage flood src=0x%x", bpf_ntohl(src_ip));
    bump_stat(STAT_DROP);
    return XDP_DROP;
}

bump_stat(STAT_PASS);
return XDP_PASS;
}
```

The only place in this function XDP_DROP can actually be returned, and it runs unconditionally after the FSM check above -- the FSM mismatch branch never short-circuits this gate, by design (see the header comment's "Enforcement policy"). Since this whole code path is already gated on label != NULL, and today that's only ever "InitialUEMessage", the explicit procedure_code == NGAP_PROC_INITIAL_UE_MESSAGE check is currently redundant with that fact, but written explicitly so the rate-limit gate stays correct and scoped to just this one procedure once ngap_procedure_label() grows to cover more codes that shouldn't share this particular rate limit. Any other outcome -- not rate-limited, or not even InitialUEMessage -- falls through to the final STAT_PASS bump and XDP_PASS.

10. xdp_ngap.c (legacy, superseded by amf_xdp.c)

The original passive-only version. Kept for reference; every parsing line here reappears in amf_xdp.c with the same logic, so this section notes only what's different or worth calling out on its own.

Lines 1-25 -- includes, constants, structs

```
#include "../vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#define ETH_P_IP 0x0800
#define IPPROTO_SCTP 132
#define SCTP_PPID_NGAP 60
#define SCTP_DATA 0

struct sctphdr_simple { __u16 source; __u16 dest; __u32 vtag; __u32 checksum; };

struct sctp_data_chunk {
    __u8 type; __u8 flags; __u16 length; __u32 ts_n;
    __u16 stream_id; __u16 stream_seq; __u32 ppid;
};
```

Identical includes, constants, and packet-structure structs to amf_xdp.c -- amf_xdp.c's versions are, in fact, copied from here.

Line 27 -- LICENSE

```
char LICENSE[] SEC("license") = "GPL";
```

Lines 29-37 -- dump_bytes (defined, never called)

```
static __always_inline void dump_bytes(void *base, void *data_end) {
    for (int i = 0; i < 32; i++) {
        if (base + i + 1 > data_end)
            break;

        __u8 b = (__u8 *)(base + i);
        bpf_printk("ngap[%d]=0x%x", i, b);
    }
}
```

A debug utility that prints up to 32 raw bytes starting at base, checking each byte's position individually against data_end before reading it -- a per-iteration bounds check, rather than one check before the loop, because the verifier can't statically prove a bulk 32-byte read is safe when the loop bound isn't tied directly to a single prior comparison. This function is defined but **never called** anywhere in this file -- leftover debug tooling from development, not part of the active code path.

Lines 39-82 -- xdp_prog: parsing down to the DATA chunk

```
SEC("xdp")
int xdp_prog(struct xdp_md *ctx)
{
    void *data = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;

    // Check ethernet header size
    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end)
        return XDP_PASS;

    // Check if the packet is IPv4
    struct iphdr *iph = (void *) (eth + 1);
    if ((void *) (iph + 1) > data_end)
        return XDP_PASS;

    // Check if the packet is SCTP
    if (iph->protocol != IPPROTO_SCTP)
        return XDP_PASS;
```

```

/* ---- Get SCTP header and data chunk ---- */
// IP header length may be > 20 bytes
void *sctp_ptr = (void *)iph + (iph->ihl * 4);

// Check SCTP header size
struct sctphdr_simple *sctp = sctp_ptr;
if ((void *)(sctp + 1) > data_end)
    return XDP_PASS;

// Check SCTP data chunk size
struct sctp_data_chunk *chunk = (void *)(sctp + 1);
if ((void *)(chunk + 1) > data_end)
    return XDP_PASS;

```

Line-for-line the same Ethernet/IP/SCTP/chunk parsing `amf_xdp.c` later reused, with one notable gap: unlike `amf_xdp.c`, **this version never checks** `eth->h_proto` -- it overlays an `iphdr` right after the Ethernet header unconditionally, assuming IPv4 rather than confirming it. In practice the subsequent `iph->protocol != IPPROTO_SCTP` check still filters out anything that isn't really IP-shaped SCTP traffic, but a non-IPv4 frame (ARP, IPv6, VLAN-tagged, etc.) would have its bytes misinterpreted as an IP header before being (most likely, but not guaranteed) rejected by that protocol check -- `amf_xdp.c`'s added `EtherType` check closes this gap.

Lines 73-101 -- procedureCode extraction and logging

```

/* Get chunk type, length, and PPID */
__u8 chunk_type = chunk->type;
__u16 chunk_len = bpf_ntohs(chunk->length);
__u32 ppid = bpf_ntohl(chunk->ppid);

if (chunk_type != SCTP_DATA)
    return XDP_PASS;

if (ppid != SCTP_PPID_NGAP)
    return XDP_PASS;

// Dump the first 32 bytes of the NGAP message
void *ngap = (void *)(chunk + 1);
if (ngap + 16 > data_end)
    return XDP_PASS;
__u8 b0 = *(__u8 *)(ngap + 0);
__u8 b1 = *(__u8 *)(ngap + 1);
__u8 b2 = *(__u8 *)(ngap + 2);

__u16 procedure_code = ((__u16)b0 << 8) | b1;
__u8 criticality = b2;

bpf_printk("SCTP sport=%d dport=%d len=%d NGAP-procedureCode=%d",
    bpf_ntohs(sctp->source),
    bpf_ntohs(sctp->dest),
    chunk_len,
    procedure_code);

return XDP_PASS;
}

```

`chunk_type`, `chunk_len`, and `ppid` are extracted eagerly as named locals here (versus `amf_xdp.c`, which inlines the equivalent comparisons directly into the if-conditions) -- `chunk_len` is computed and used only in the final `printk`, never in a bounds decision. The DATA-chunk and NGAP-PPID filters are the same two checks `amf_xdp.c` keeps. A third byte, `b2`, is read here and assigned to `criticality` -- computed but then **never used** (not even in the final `printk`) -- a piece of dead extraction `amf_xdp.c` dropped when it only ended up needing the two `procedureCode` bytes. The function's entire payload is one `bpf_printk` call logging source/dest port, chunk length, and procedure code -- pure observation. The final `return XDP_PASS` is unconditional and is the only possible outcome anywhere in this function -- there is no drop path at all, which is

the one-sentence summary of why this file was superseded: it can observe NGAP traffic but can never act on what it observes.

11. Makefile

Builds only the uprobe/tracepoint tracer. Neither `amf_xdp.c` nor `xdp_ngap.c` has a rule here -- both are hand-compiled per the command blocks documented in their own file headers / the README.

Lines 1-8 -- name and tool variables

```
NF := amf

CLANG := clang
CC := gcc
BPFTOOL := bpftool

ARCH := $(shell uname -m | sed 's/x86_64/x86/')
KERNEL_SRC := /lib/modules/$(shell uname -r)/build
```

NF ("network function") is used as a filename prefix throughout the rest of the file (`$(NF)_tracer.bpf.c`, `$(NF)_loader`, etc.) -- the only line that would need changing to reuse this Makefile's structure for a different NF's tracer. ARCH shells out to `uname -m` and rewrites `x86_64` to `x86`, matching the short architecture name `libbpf/vmlinux.h` conventions expect in the `__TARGET_ARCH_*` macro. `KERNEL_SRC` points at the currently-running kernel's build directory, used below to add its UAPI headers to the include path.

Lines 10-14 -- compiler flags

```
CFLAGS := -O2 -g -Wall
BPF_CFLAGS := -O2 -g -target bpf -D__TARGET_ARCH_$(ARCH)

INCLUDES := -I. -I. -I$(KERNEL_SRC)/usr/include
LIBS := -lbpf -lelf -lz
```

`CFLAGS` for the ordinary userspace gcc compiles; `BPF_CFLAGS` adds clang's `-target bpf` cross-compilation flag plus the architecture macro for the BPF compiles. `INCLUDES` covers this directory (for sibling headers like `amf_comm.h`), the parent directory (`common.h`, `events.h`, `vmlinux.h`), and the running kernel's own headers. `LIBS` links against `libbpf` and its own dependencies (`libelf` for ELF parsing, `zlib` for compressed BTF).

Lines 16-25 -- file/target variables

```
BPF_SRC := $(NF)_tracer.bpf.c
BPF_OBJ := $(NF)_tracer.bpf.o
SKEL_HDR := $(NF)_tracer.skel.h

USER_SRCS := $(NF)_loader.c $(NF)_comm.c $(NF)_functions.c
USER_OBJS := $(USER_SRCS:.c=.o)

COMMON_OBJ := ../common.o

TARGET := $(NF)_loader
```

Names every file this build touches, in terms of `$(NF)`: the single BPF source, its compiled object, and the generated skeleton header; the three userspace sources, with `USER_OBJS` derived from them via Make's `$(VAR:.c=.o)` substitution (turning each `.c` name into the matching `.o`); the shared common object (built once in the parent directory, reused by every NF tracer in the wider repo, not just this one); and the final binary name.

Lines 27-29 -- .PHONY and default target

```
.PHONY: all clean

all: $(TARGET)
```

`.PHONY` tells Make that `all` and `clean` are recipe names, not files to check timestamps on (protects against a stray file literally named "clean" silently breaking the rule). `all`, the first target defined and therefore Make's default goal, simply depends on the final binary.

Lines 31-36 -- common object + link rule

```
# Build the shared common object from the parent directory.
$(COMMON_OBJ): ../common.c ../common.h
$(CC) $(CFLAGS) -I.. -c $< -o $@

$(TARGET): $(SKEL_HDR) $(USER_OBJS) $(COMMON_OBJ)
$(CC) $(CFLAGS) $(USER_OBJS) $(COMMON_OBJ) -o $@ $(LIBS)
```

The common-object rule compiles `../common.c` whenever it or its header changes. The link rule for the final target depends on the skeleton header (must exist and be current before `amf_loader.c`, which includes it, can even compile), the three user objects, and the common object -- `$</$@` are Make's automatic variables for "first prerequisite" and "target name", used throughout so the rules stay generic rather than repeating literal filenames.

Lines 38-39 -- generic compile rule

```
%.o: %.c
$(CC) $(CFLAGS) $(INCLUDES) -c $< -o $@
```

A pattern rule matching any `.c/.o` pair -- this single rule covers compiling all three `USER_OBJS` without a separate explicit rule per file.

Lines 41-45 -- BPF object and skeleton rules

```
$(BPF_OBJ): $(BPF_SRC) ../vmlinux.h
$(CLANG) $(BPF_CFLAGS) $(INCLUDES) -c $< -o $@

$(SKEL_HDR): $(BPF_OBJ)
$(BPFTOOL) gen skeleton $< > $@
```

The BPF object depends on both its source and `vmlinux.h` (so a kernel/BTF header change triggers a rebuild too, even if `amf_tracer.bpf.c` itself didn't change), compiled with clang's BPF target flags. The skeleton header is generated from the compiled object via `bpftool gen skeleton` -- this is the step that produces `struct amf_tracer_bpf` and its `open/load/attach/destroy` functions, by introspecting the object's embedded BTF for every map and program name.

Lines 47-48 -- clean

```
clean:
rm -f $(TARGET) $(USER_OBJS) $(BPF_OBJ) $(SKEL_HDR) $(COMMON_OBJ)
```

Removes every generated artifact this Makefile produces -- notably, does not remove any hand-compiled `amf_xdp.o` or its pinned BPF maps under `/sys/fs/bpf`, since neither is a Makefile-managed artifact.